# xmarkdown2pdf — Architecture Design

> Architect: GitHub Copilot
> Date: 2026-03-14
> Version: 1.0

## 1. Overview

`xmarkdown2pdf` is a VS Code extension that converts Markdown files (including embedded Mermaid and PlantUML diagrams) to **HTML** or **PDF** locally, with a live **WYSIWYG preview** panel. All heavy rendering is done in-process or via local tools — no cloud dependency by default.

```
┌──────────────────────────────────────────────────────────────┐
│                          VS Code                             │
│  ┌──────────────┐   commands   ┌──────────────────────────┐  │
│  │  Editor      │──────────────▶│  Extension Host (Node)   │  │
│  │ (*.md)       │◀──────────────│  xmarkdown2pdf           │  │
│  └──────────────┘   webview    └──────────┬───────────────┘  │
│                                           │                  │
│          ┌────────────────────────────────┼──────────┐       │
│          ▼                                ▼          ▼       │
│   MarkdownPipeline           HtmlExporter   PdfExporter     │
│   (markdown-it + plugins)                  (Puppeteer)       │
│          │                                                   │
│   ┌──────┼──────┐                                            │
│   ▼      ▼      ▼                                            │
│ Core  Mermaid PlantUML                                       │
│       Plugin   Plugin                                        │
└──────────────────────────────────────────────────────────────┘
```

## 2. Requirements Mapping

| Requirement | Solution |
|---|---|
| VS Code extension | Standard `vscode` Extension API, TypeScript |
| Local conversion of Markdown + Mermaid + PlantUML | `markdown-it` pipeline + renderer plugins |
| Export to HTML | `HtmlExporter` command |
| Export to PDF | `PdfExporter` via Puppeteer (headless Chromium) |
| WYSIWYG preview | VS Code `WebviewPanel` with live document sync |
| Flexible library upgrades | `LibManager + libs.json` version manifest |

## 3. Tech Stack

| Layer | Library | Rationale |
|---|---|---|
| Markdown parsing | `markdown-it` | Plugin-based, same as VS Code built-in |
| Mermaid (preview) | `mermaid.min.js` in WebView | Native browser rendering, zero extra process |
| Mermaid (export) | Puppeteer renders the HTML with mermaid.js | Reuses same path as preview export |
| PlantUML | `node-plantuml` (wraps plantuml.jar + Java) | Local, offline; Java is optional — falls back to configurable server URL |

| Layer | Library | Rationale |
|---|---|---|
| PDF export | `puppeteer` (bundled Chromium) | Pixel-perfect HTML → PDF; handles SVG, CSS |
| HTML export | `markdown-it` pipeline output | Direct string output, no extra dependency |
| Language | TypeScript | Type safety, VS Code ecosystem standard |
| Bundler | `esbuild` (via VS Code extension template) | Fast, minimal bundle for distribution |

---

## 4. Project Structure

```
xmarkdown2pdf/
├── package.json              # Extension manifest, contributes,
dependencies
├── tsconfig.json
├── esbuild.js                # Build script
├── libs.json                 # External library version manifest
(upgradeable)
│
├── src/
│   ├── extension.ts          # activate() / deactivate(), registers all
commands
│   │
│   ├── commands/
│   │   ├── exportHtml.ts     # Command: xmarkdown2pdf.exportHtml
│   │   ├── exportPdf.ts      # Command: xmarkdown2pdf.exportPdf
│   │   └── openPreview.ts    # Command: xmarkdown2pdf.openPreview
│   │
│   ├── renderer/
```

```
│   │   ├── MarkdownPipeline.ts    # Builds & caches the markdown-it instance
+ plugins
│   │   ├── RendererPlugin.ts      # Interface: { name, apply(md:
MarkdownIt): void }
│   │   └── plugins/
│   │       ├── MermaidPlugin.ts   # Fences mermaid → <div class="mermaid">
│   │       └── PlantUmlPlugin.ts  # Fences plantuml → inline SVG (via node-
plantuml)
│   │
│   ├── preview/
│   │   ├── PreviewPanel.ts        # Singleton WebviewPanel, watches doc
changes
│   │   └── previewTemplate.ts     # Builds full HTML page for the webview
│   │
│   ├── exporter/
│   │   ├── HtmlExporter.ts        # Renders pipeline → writes .html file
│   │   └── PdfExporter.ts         # Launches Puppeteer, prints .pdf file
│   │
│   ├── libs/
│   │   └── LibManager.ts          # Reads libs.json, downloads/updates
versioned libs
│   │
│   └── config/
│       └── Settings.ts            # Typed wrapper around
vscode.workspace.getConfiguration
│
├── media/
│   ├── preview.css            # Base styles injected into WebView & HTML
export
│   └── libs/                  # Runtime JS libs (managed by LibManager)
│       ├── mermaid.min.js     # Bundled at install; replaced by LibManager
on upgrade
│       └── highlight.min.js   # Syntax highlighting
│
```

```
└── test/
    ├── suite/
    │   ├── renderer.test.ts
    │   └── exporter.test.ts
    └── runTest.ts
```

---

# 5. Component Details

## 5.1 `MarkdownPipeline`

Central singleton that assembles `markdown-it` with all plugins. New diagram types can be added by registering a new `RendererPlugin` — no core changes needed.

```
MarkdownPipeline
  .register(new MermaidPlugin())
  .register(new PlantUmlPlugin())
  .render(markdownText): Promise<string>  // returns HTML fragment
```

**Plugin Interface:**

```
interface RendererPlugin {
  name: string;
  apply(md: MarkdownIt): void;   // modify the markdown-it instance
}
```

## 5.2 `MermaidPlugin`

- Intercepts fenced code blocks with language `mermaid`
- In preview mode: emits `<div class="mermaid">…</div>` — mermaid.js in the WebView renders it client-side at runtime
- In export mode: the Puppeteer-rendered page already contains rendered SVGs (since mermaid.js runs in the headless browser)

- Zero extra process or server

### 5.3 `PlantUmlPlugin`

- Intercepts fenced code blocks with language `plantuml`
- Renders to SVG via `node-plantuml` (spawns local `java -jar plantuml.jar`)
- **Fallback chain** (configured via settings):
  1. Local Java + plantuml.jar (offline)
  2. Self-hosted PlantUML server URL
  3. Kroki.io public endpoint (online fallback)
- The jar path is configurable; `LibManager` can auto-download the latest jar

### 5.4 `PreviewPanel` (WYSIWYG)

```
VS Code Editor ──onChange──▶ PreviewPanel.update(doc.getText())
                                  │
                             MarkdownPipeline.render()
                                  │
                             WebviewPanel.postMessage({ html })
                                  │
                             previewTemplate.html receives message
                             and updates innerHTML of #content
```

- Panel lives in `vscode.ViewColumn.Beside`
- Scroll sync: editor cursor position → webview scroll via `postMessage`
- Security: `webview.options.localResourceRoots` restricts to extension `media/` only; no `allowScripts` on untrusted content areas

### 5.5 `PdfExporter`

```
// Sequence:
// 1. Render markdown to full HTML page (with mermaid.js script embedded)
// 2. Launch Puppeteer (bundled Chromium — no external install needed)
// 3. page.setContent(html) — let mermaid.js render diagrams
```

```
// 4. page.waitForSelector('.mermaid[data-processed]') — wait for render
// 5. page.pdf({ path, format, printBackground: true })
// 6. Close browser
```

Puppeteer is listed as a regular dependency; its bundled Chromium is downloaded at `npm install` time (controlled by `PUPPETEER_SKIP_CHROMIUM_DOWNLOAD` env var for CI).

## 5.6 `LibManager` — Flexible Upgrades

`libs.json` is the single source of truth for external library versions:

```
{
  "mermaid": {
    "version": "11.4.1",
    "cdn":
"https://cdn.jsdelivr.net/npm/mermaid@{version}/dist/mermaid.min.js",
    "localPath": "media/libs/mermaid.min.js"
  },
  "plantuml": {
    "version": "1.2025.2",
    "downloadUrl":
"https://github.com/plantuml/plantuml/releases/download/v{version}/plantuml-
{version}.jar",
    "localPath": "media/libs/plantuml.jar"
  },
  "highlight": {
    "version": "11.10.0",
    "cdn":
"https://cdn.jsdelivr.net/npm/highlight.js@{version}/build/highlight.min.js",
    "localPath": "media/libs/highlight.min.js"
  }
}
```

Command **`xmarkdown2pdf.upgradeLibs`**:

1. Reads `libs.json`
2. Queries npm registry / GitHub releases for latest versions
3. Downloads new files to `media/libs/`
4. Updates version fields in `libs.json`
5. Reports results in an output channel

---

## 6. Commands (package.json contributes)

| Command ID | Title | Keybinding |
|---|---|---|
| `xmarkdown2pdf.openPreview` | Markdown: Open Preview | `Ctrl+Shift+V` |
| `xmarkdown2pdf.exportHtml` | Markdown: Export to HTML | — |
| `xmarkdown2pdf.exportPdf` | Markdown: Export to PDF | — |
| `xmarkdown2pdf.upgradeLibs` | Markdown: Upgrade Libraries | — |

All commands are only active when `editorLangId == markdown`.

---

## 7. Configuration Settings

```
// In package.json contributes.configuration
{
  "xmarkdown2pdf.pdf.format": "A4",           // A3, A4, Letter, Legal
  "xmarkdown2pdf.pdf.margin": { "top": "20mm", "bottom": "20mm", ... },
  "xmarkdown2pdf.pdf.printBackground": true,
  "xmarkdown2pdf.plantuml.renderMode": "local",  // "local" | "server" |
"kroki"
  "xmarkdown2pdf.plantuml.serverUrl": "",
  "xmarkdown2pdf.plantuml.jarPath": "",          // auto-resolved if empty
  "xmarkdown2pdf.preview.scrollSync": true,
  "xmarkdown2pdf.preview.theme": "github",       // "github" | "dark" |
"custom"
```

```
  "xmarkdown2pdf.preview.customCssPath": ""
}
```

---

# 8. Data Flow Diagrams

## Export to PDF

```
User triggers "Export PDF"
        │
        ▼
exportPdf.ts
  ├─ Gets active editor document text
  ├─ Resolves output path (same dir, .pdf extension)
  ├─ Calls PdfExporter.export(text, outputPath, settings)
  │       │
  │       ▼
  │   MarkdownPipeline.render(text)
  │       ├─ MermaidPlugin  → <div class="mermaid">…</div>
  │       └─ PlantUmlPlugin → <svg>…</svg> (inline)
  │       └─ returns HTML fragment
  │
  │   previewTemplate.build(fragment, { embedScripts: true })
  │       └─ inlines mermaid.min.js + CSS → full HTML page
  │
  │   Puppeteer
  │       ├─ page.setContent(fullHtml)
  │       ├─ mermaid.initialize + mermaid.run() (in-page)
  │       ├─ waitForSelector('.mermaid svg')
  │       └─ page.pdf(outputPath)
  │
  └─ Shows "Saved to …" notification
```

### WYSIWYG Preview

```
Document onChange (debounced 300ms)
        │
        ▼
PreviewPanel.update(text)
        │
        ▼
MarkdownPipeline.render(text)
        │
        ▼
webview.postMessage({ type: 'update', html })
        │
        ▼
previewTemplate.js (in webview)
  ├─ document.getElementById('content').innerHTML = html
  └─ mermaid.run()   ← re-renders all .mermaid divs
```

---

## 9. Security Considerations

- WebView `localResourceRoots` is restricted to `media/` only
- PlantUML server URL is validated to be a proper `http(s)://` URL before use
- Puppeteer runs in a sandboxed headless Chrome process (no `--no-sandbox` unless Linux CI)
- No arbitrary code execution from document content; diagram sources are passed as strings to renderers only
- `libs.json` download URLs are validated against an allowlist before fetch

---

## 10. Dependency Budget

```
dependencies (shipped):
  markdown-it          ~400 KB   Core parser
  node-plantuml        ~50 KB    PlantUML wrapper (plantuml.jar separate)
  puppeteer            ~7 MB     headless Chromium (downloaded separately)

devDependencies:
  @types/vscode
  @types/node
  typescript
  esbuild
  @vscode/test-electron

media/libs (managed by LibManager):
  mermaid.min.js       ~3 MB    Client-side diagram renderer
  highlight.min.js     ~1 MB    Syntax highlighting
  plantuml.jar         ~10 MB   Local PlantUML renderer
```

Puppeteer's Chromium (~170 MB) is downloaded once at `npm install` and cached by VS Code extension infra. Users who cannot allow this can set `"xmarkdown2pdf.plantuml.renderMode": "server"` and avoid Puppeteer by setting `"xmarkdown2pdf.pdf.engine": "wkhtmltopdf"` (future option).

---

## 11. Extensibility & Upgrade Path

The plugin registry pattern means adding support for a new diagram type (e.g., D2, Graphviz) is done by:

1. Creating `src/renderer/plugins/NewDiagramPlugin.ts` implementing `RendererPlugin`
2. Registering it in `MarkdownPipeline` constructor

Library upgrades are fully automated via the `upgradeLibs` command — no code changes needed to pick up a new mermaid or plantuml version.

## 12. Implementation Phases

| Phase | Deliverables |
|---|---|
| **Phase 1 — Core** | `MarkdownPipeline`, `MermaidPlugin`, `HtmlExporter`, basic settings |
| **Phase 2 — PlantUML** | `PlantUmlPlugin`, `LibManager`, `libs.json`, `upgradeLibs` command |
| **Phase 3 — PDF** | `PdfExporter` (Puppeteer), PDF settings |
| **Phase 4 — Preview** | `PreviewPanel`, scroll sync, theme support |
| **Phase 5 — Polish** | Packaging, VSIX publish, CI/CD |